

Creating Valid Triples

- When we have some valid triple(s) given to us, how can we use them to create more valid triple(s)? Note that, **the validity here can be under either correctness**: if partially correct triples are provided, then we can use them to create another partially correct triple; if totally correct triples are provided, then we can use them to create another totally correct triple.
1. If we are given two valid triples, can we sequence them?
 - a. We have valid triples $\{x = k\} S_1 \{x = m\}$, and $\{x = m\} S_2 \{x = n\}$, what can be a postcondition for $\{x = k\} S_1; S_2 \{q\}$?
It is quite easy to see that $\{x = k\} S_1; S_2 \{x = n\}$ can be a valid triple.
 - **[Sequence Rule]** If we have valid triples $\{p\} S_1 \{q\}$ and $\{q\} S_2 \{r\}$, then we have valid triple $\{p\} S_1; S_2 \{r\}$.
 - b. What if we have triples $\{x = k\} S_1 \{x \geq m\}$ and $\{x \geq m - 1\} S_2 \{x = n\}$, can we still combine these two triples into $\{x = k\} S_1; S_2 \{x = n\}$?
Yes, since after executing S_1 we will end up some state(s) $\tau \models x \geq m$, so τ also satisfies the precondition of S_2 .
 - **[Extended Sequence Rule]** If we have valid triples $\{p\} S_1 \{q\}$ and $\{q'\} S_2 \{r\}$, and $q \Rightarrow q'$, then we have valid triple $\{p\} S_1; S_2 \{r\}$.
2. Let $\{x \geq 0\} S \{y < 0\}$ be a valid triple.
 - a. Is $\{x \geq 5\} S \{y < 0\}$ valid?
Yes. $x \geq 5$ is a subcollection of $x \geq 0$, if S works “well” on all states satisfying $x \geq 0$ then it also works well on a state satisfying $x \geq 5$.
 - **[Strengthening Precondition]** Strengthening the precondition of valid triple doesn’t affect its validity.
 - b. Is $\{x \geq -5\} S \{y < 0\}$ valid?
We cannot decide, since we don’t know anything about the execution of S in a state σ with $-5 \leq \sigma(x) < 0$.
Weakening the precondition of a valid tripe can affect its validity.
 - c. Is $\{x \geq 0\} S \{y \leq 0\}$ valid ?
Yes. $y < 0$ is a subcollection of $y \leq 0$ (since $y \leq 0$ means $y < 0 \vee y = 0$), If S terminates in states satisfying $y < 0$ then those states also satisfying $y \leq 0$.
 - **[Weakening Postcondition]** Weakening the postcondition of valid triple doesn’t affect its validity.
 - d. Is $\{x \geq 0\} S \{y < -5\}$ valid ?
We cannot decide, since we only know the execution of S terminate in states satisfying $y < 0$, but we don’t know whether those states satisfy $y < -5$. Strengthening the postcondition of a valid tripe can affect its validity.
 - e. Among valid triples $\{x \geq 0\} S \{y < 0\}$, $\{x \geq 5\} S \{y < 0\}$, and $\{x \geq 0\} S \{y \leq 0\}$, which triple gives us the most information?

- Compare $\{x \geq 0\} S \{y < 0\}$ and $\{x \geq 5\} S \{y < 0\}$. The previous one tells us that S can work well whenever $x \geq 0$; the later one says S can work well ONLY when $x \geq 5$. The previous one contains more information.
- Compare $\{x \geq 0\} S \{y < 0\}$ and $\{x \geq 0\} S \{y \leq 0\}$. The previous one tells us that S can provide us an outcome with $y < 0$; the later one says S can provide us a not-so-accurate outcome with $y < 0$ or $y = 0$. The previous one contains more information.
- In general, weakening the postcondition or strengthening the prediction makes a valid triple to *lose information* and become less useful. On the other hand, weakening the prediction or strengthening the postcondition might affect the validity of a triple. Thus, it is important to find **the weakest precondition** and/or **the strongest postcondition** (and maintaining the validity at the same time), to create the “good” triples.

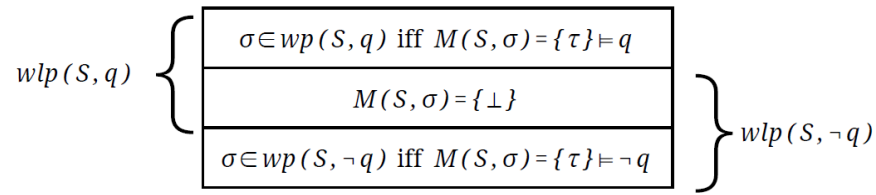
Weakest Preconditions

- w is the **weakest precondition of S and q** (we write $w = wp(S, q)$ or $w \Leftrightarrow wp(S, q)$) if w is a precondition for S and q such that $\{w\} S \{q\}$ is **totally valid** and w can't be weakened anymore.
 - In other words, w is the weakest precondition of S and q means “ $\models_{tot} \{w\} S \{q\}$ and there exists no predicate r weaker than w such that $\models_{tot} \{r\} S \{q\}$ ”.
 - In terms of collection of states: $wp(S, q) = \{\sigma \in \Sigma \mid M(S, \sigma) \models q\}$.
- 3. Let's consider $w = wp(x := x + 1, x \geq 2)$.
If we use terms of states, we can see w is the collection of all σ that makes $M(x := x + 1, \sigma) \models (x \geq 2)$. This collection containing states such as $\{x = 5\}, \{x = 1, y = 3\}, \{x = 100, z = 1, y = 4\}$... In general, we can say that is the collection of states that satisfy $x \geq 1$.
- 4. Decide whether it is true or false for the following statements.
 - a. If $\models_{tot} \{r\} S \{q\}$, then $r \Rightarrow wp(S, q)$. True.
 - b. If $r \Rightarrow wp(S, q)$, then $\models_{tot} \{r\} S \{q\}$. True.
 - We can say that $\models_{tot} \{r\} S \{q\}$ if and only if $r \Rightarrow wp(S, q)$.
 - c. If $\sigma \not\models wp(S, q)$, then we know nothing interesting about $M(S, \sigma)$.
False. If $\sigma \not\models wp(S, q)$, or $\sigma \notin wp(S, q)$; then we know that $M(S, \sigma) \not\models q$.
 - d. Assuming that q won't be evaluated to $\perp$ in any state. If S is deterministic, then $\models \{\neg wp(S, q)\} S \{\neg q\}$.
True. For any state σ , if $\sigma \models \neg wp(S, q)$ and $M(S, \sigma) = \{\tau\}$, we must have $\tau \not\models q$; in other words, $\tau = \perp$ or $\tau \models \neg q$; thus $M(S, \sigma) \models \neg q$.
 - e. If $u \Leftrightarrow wp(S, q)$, then u is also the weakest precondition of S and q .
True. For example, if $wp(S, q)$ is $x \geq 1$, then $x > 0$ or $1 \leq x$ can also be used as the weakest precondition.
- The **weakest liberal precondition for S and q** , written $wlp(S, q)$, is like $wp(S, q)$ but for partial correctness.
 - In other words, w is the weakest liberal precondition of S and q means “ $\models \{w\} S \{q\}$ and there exists no predicate r that is weaker than w such that $\models \{r\} S \{q\}$ ”.
 - In terms of collection of states: $wlp(S, q) = \{\sigma \in \Sigma \mid M(S, \sigma) \neq \perp \models q\}$.
 - We can say that $\models \{r\} S \{q\}$ if and only if $r \Rightarrow wlp(S, q)$.

- We care about wp and wlp since they are the most general conditions a program requires to run “successfully” in when we want to get a certain postcondition.
 - From one of the above examples, we learned that if a state σ does not satisfy wp , then it is guaranteed that $M(S, \sigma) \not\models q$. Similarly, for wlp , if $\sigma \notin wlp$, then $M(S, \sigma) \not\models q$.

(wp and wlp for deterministic program)

- The following figure illustrates the relationships between wp and wlp for **deterministic** programs (here, we assume that $\tau(q) \neq \perp$ if $\tau \neq \perp$). What actually needs here is that $M(S, \sigma)$ contains exactly one (possibly pseudo) state.



- For a state σ and a deterministic program S , and a postcondition q (remember that we assume that $\tau(q) \neq \perp$ if $\tau \neq \perp$), we can have three possible outcomes for $M(S, \sigma)$:
 - 1) $M(S, \sigma) = \{\tau\}$ and $\{\tau\} \models q$.
 - 2) $M(S, \sigma) = \{\perp\}$.
 - 3) $M(S, \sigma) = \{\tau\}$ and $\{\tau\} \models \neg q$.
- $wp(S, q)$ is set of all σ in situation 1).
- $wp(S, \neg q)$ is set of all σ in situation 3).
- $wlp(S, q)$ is set of all σ in situation 1) and 2).
- $wlp(S, \neg q)$ is set of all σ in situation 2) and 3).

5. True or False.

- Let S be deterministic. $wlp(S, T) \Leftrightarrow T$.
True. For any state σ , either $M(S, \sigma) = \perp$ or $M(S, \sigma) \neq \perp$. If $M(S, \sigma) = \perp$, then $\sigma \in wlp(S, T)$; if $M(S, \sigma) \neq \perp$, then $M(S, \sigma) \models T$. Thus, all states satisfy $wlp(S, T)$; in other words, $wlp(S, T) \Leftrightarrow T$.
- Let S be deterministic. $wp(S, F) \Leftrightarrow F$.
True. For any state σ , either $M(S, \sigma) = \perp$ or $M(S, \sigma) \neq \perp$. If $M(S, \sigma) = \perp$, then $\sigma \notin wp(S, F)$; if $M(S, \sigma) \neq \perp$, then $M(S, \sigma) \not\models F$. Thus, $wp(S, F)$ is an empty set; in other words, $wp(S, F) \Leftrightarrow F$.
- $wp(y := x * x, y \geq 4) \Leftrightarrow wlp(y := x * x, y \geq 4)$
True, because the statement $y := x * x$ is loop-free and cannot create a runtime error, the postcondition $y \geq 4$ cannot be evaluated to $\perp$. As an aside, using *backward assignment* rule, we can get $wp(y := x * x, y \geq 4) \Leftrightarrow x * x \geq 4$.